

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



PROJECT EMBEDDED SYSTEM

TOPIC:
DESIGN TWO-LED DICE GAME USING STM32

CC01 --- GROUP 11 --- SES 252

INSTRUCTOR: Bùi Quốc Bảo

Students	Student IDs	Grade
Nguyễn Gia Huy	2311179	
Phạm Vĩnh Phát	2351059	

Table of contents

Chapter 1 Introduction	3
1.1 General ideas	3
1.2 Game rules.....	3
<i>1.2.1 Setup and gameplay</i>	<i>3</i>
<i>1.2.2 Match resolution</i>	<i>3</i>
1.3 Hardware components	4
Chapter 2 Requirements	5
2.1 Functional Requirements	5
2.2 Non-Functional & Constraints	5
Chapter 3 Design	6
3.1 Hardware design	6
3.2 Power supply design.....	6
3.3 Microcontroller design	7
3.4 LEDs arranged in 7-pip pattern	8
3.5 LCD 1602 and Buzzer	9
3.6 Hardware Implementation.....	10
Chapter 4 Software design.....	11
4.1 Full program.....	11
Chapter 5 Result	16

Chapter 1 Introduction

1.1 General ideas

This project aims to create a simple entertainment device where the rules involve comparing the results of dice rolls between two players to determine the winner.

The "STM32 2-Player Electronic Dice Game" project is an interactive turn-based embedded system developed to simulate the traditional dice game for two players. The system is built on the STM32 Nucleo microcontroller platform combined with peripheral electronic components to provide a vivid and engaging experience through both visual (LED, LCD) and auditory (buzzer) input.

1.2 Game rules

1.2.1 Setup and gameplay

The system is programmed with a set of rules to calculate scores and automatically determine the outcome to decide the winner and loser between two players. This competitive game is designed for exactly two players, with player 1 always going first and player 2 going second. Therefore, a single match only lasts for 2 turns, the "RESET" button should be pressed to begin another match.

The players observe the LCD screen and wait until it shows that it is their turn, press and hold the "ROLL" button to create a random dice roll effect. When the button is released, the result is locked and displayed via LED lights and also on the screen. The total of dots on two dices is the base score that player has. When a game ends, the system will play the corresponding Buzzer melody (Win, Lose, Draw), the game state is temporarily locked (the "ROLL" button is disabled to avoid interference), to start a new game, the user must press the "RESET" button. The LCD and LED will be cleared up to prepare for the new state.

In order to make the game more competitive and different from normal dice games, we also developed some new rules for the system:

- Auto-Loss: when a player rolled a 1-1 or 6-6, they will automatically lose the game.
- Auto-Win: when a player rolled a double as 2-2, 3-3, 4-4 or 5-5, they will automatically win the game.

Note: Regardless of the outcome Player 1 rolls (whether Auto-Win or Auto-Loss), Player 2 always gets to take their turn.

1.2.2 Match resolution

After both players have rolled their dice, STM32 will compare the logic as follows:

- If Player 1 rolls an Auto-Loss: Player 2 wins. However, if Player 2 also rolls an Auto-Loss, the match is a draw (DRAW).

- If Player 1 rolls an Auto-Win: Player 1 wins. If Player 2 also rolls an Auto-Win, the match is a draw (DRAW).
- Special case for Player 2: If Player 1 does not roll a pair, but Player 2 rolls an Auto-Win, Player 2 wins; if Player 2 rolls an Auto-Loss, Player 1 wins.
- Normal determination: If neither player rolls a pair, the system compares the total points. The player with the higher score wins. A tie is a draw (DRAW).

1.3 Hardware components

The system uses basic and common electronic components, connected to a central microcontroller to execute the game logic. The main components include:

- Central Microcontroller: 1x STM32 Nucleo board (F401RE series).
- Information Display Block: 1x 16x2 Character LCD screen to display turns, scores, and match results.
- Dice Display Block: 14x Single LEDs (Red, 5mm) arranged in a 7-pip pattern for each die, combined with 220-ohm resistors for circuit protection.
- Sound Block: 1x Passive Buzzer (5V) creating 8-bit retro-style sound effects for events (Win, Loss, Draw).
- User interface block: two Push buttons for ROLL and RESET functions, using 10k-ohm pull-up resistors.

The purpose of using 14 LEDs for two dice is to make a 7-pip pattern so that when the LED lights up, it will display a shape similar to the number on a regular die.

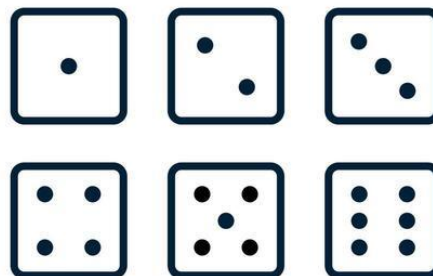


Figure 1.1: 7-pip pattern on a dice

Chapter 2 Requirements

2.1 Functional Requirements

Number	Requirements
2.1.1	The system is capable of generating valid dice face values (from 1 to 6) instantly when the player manipulates it.
2.1.2	Using 14 LEDs (arranged in the standard 7-dot pattern of a dice) to display the dice faces, combined with a 16x2 LCD screen to monitor the game status in real time.
2.1.3	Equipped with an buzzer to emit corresponding sounds when the dice are rolled, or when a Win/Tie result is obtained.
2.1.4	A hardware button is used to allow users to clear the current game state and start a new one.

Table 1: Functional requirements

2.2 Non-Functional & Constraints

Number	Requirements
2.2.1	The system has an extremely fast response time with an output signal delay always below 100ms.
2.2.2	Use a hardware RC debouncing circuit with capacitors and resistors to ensure that the input signal from the buttons remains stable and free from interference.
2.2.3	The 5V power supply from the USB port will pass through a voltage regulator IC to reduce it to 3.3V, which is then supplied to the microcontroller. The circuit operates safely at 3.3V, and the LEDs are strictly protected by a 220 Ohm current-limiting resistor and use N-MOSFETs to switch the current on/off.

Table 2: Non-Functional & Constraints

Chapter 3 Design

3.1 Hardware design

Due to its simple and minimalist design, this entertainment system doesn't require extensive hardware. As listed above, the main components are divided into several blocks and connected to the MCU via various protocols. Most components are connected via the MCU's GPIO pins; the LCD1602 uses four GPIO pins, the USB port provides 5V power to the MCU, and the ST-Link allows programming from the STM32CubeIDE into the MCU.

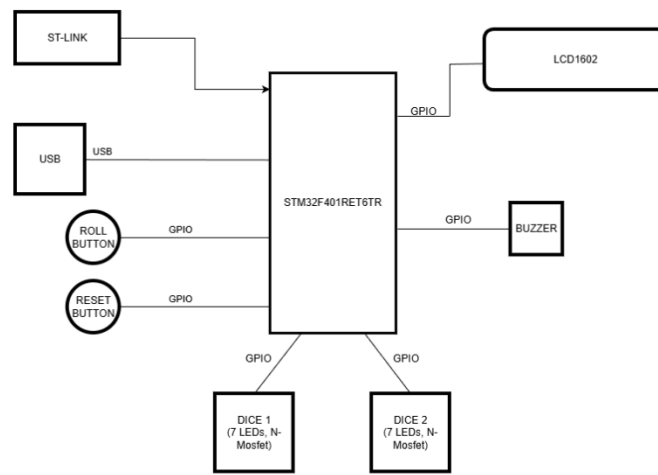


Figure 3.1: Basic block diagram for main system

3.2 Power supply design

The 5V voltage from the laptop's USB port will be the primary power source for the system.

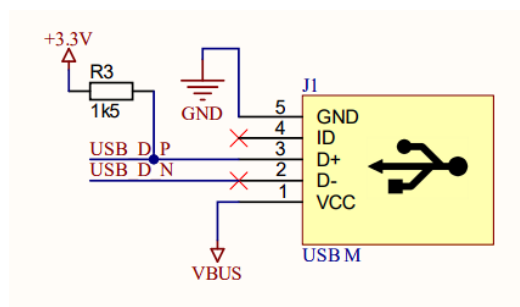


Figure 3.2: USB for power supply

In fact, most STM32 microcontrollers use a 3.3V power supply, with a maximum acceptable voltage of 3.6V. Therefore, applying a 5V output from a laptop's USB port is too high and could cause circuit damage. To ensure safety, the system uses a voltage regulator to reduce the voltage from 5V to a suitable 3.3V for the MCU to operate.

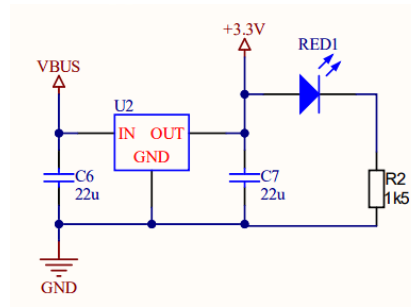


Figure 3.3: The LDO steps down the voltage from 5V to 3.3V.

This regulator is integrated into the STM32 Nucleo Board F401RE that the team used. Here, the system distributes voltage to power components that require different voltage sources.

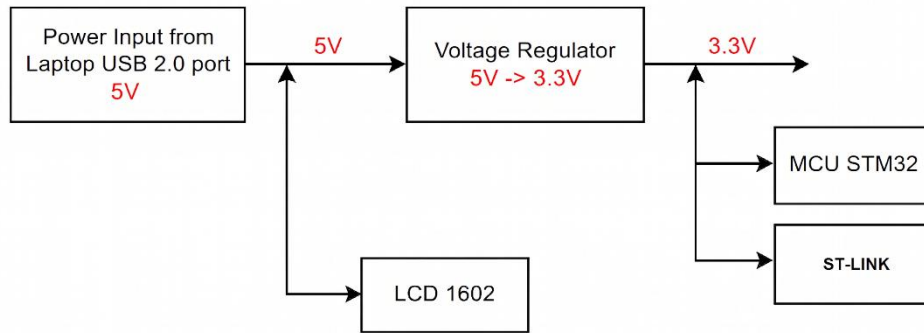


Figure 3.4: Power tree for components

3.3 Microcontroller design

The Central Microcontroller Unit (MCU) acts as the "brain" coordinating all system operations, using an STM32 microcontroller (specifically the STM32F401RET6TR). In terms of hardware connectivity, this unit is powered via the VDD and GND pins, uses a series of general-purpose GPIO ports to communicate directly with all peripheral modules, and integrates dedicated SWDIO/SWCLK pins for programming and debugging.

Functionally, the MCU handles the entire real-time logic processing of the game. Specifically, the microcontroller continuously scans signals from input buttons (ROLL, RESET), executes algorithms to generate random numbers for the two dice, and analyzes the win/loss results based on pre-programmed rules (including the Auto-Win/Auto-Loss double-sided rules). Immediately after processing, the MCU will output a voltage signal to precisely turn on/off 14 LEDs simulating dice faces, send match status data to the 1602 LCD screen, and simultaneously send PWM pulses to the buzzer to create vivid sound effects for player feedback

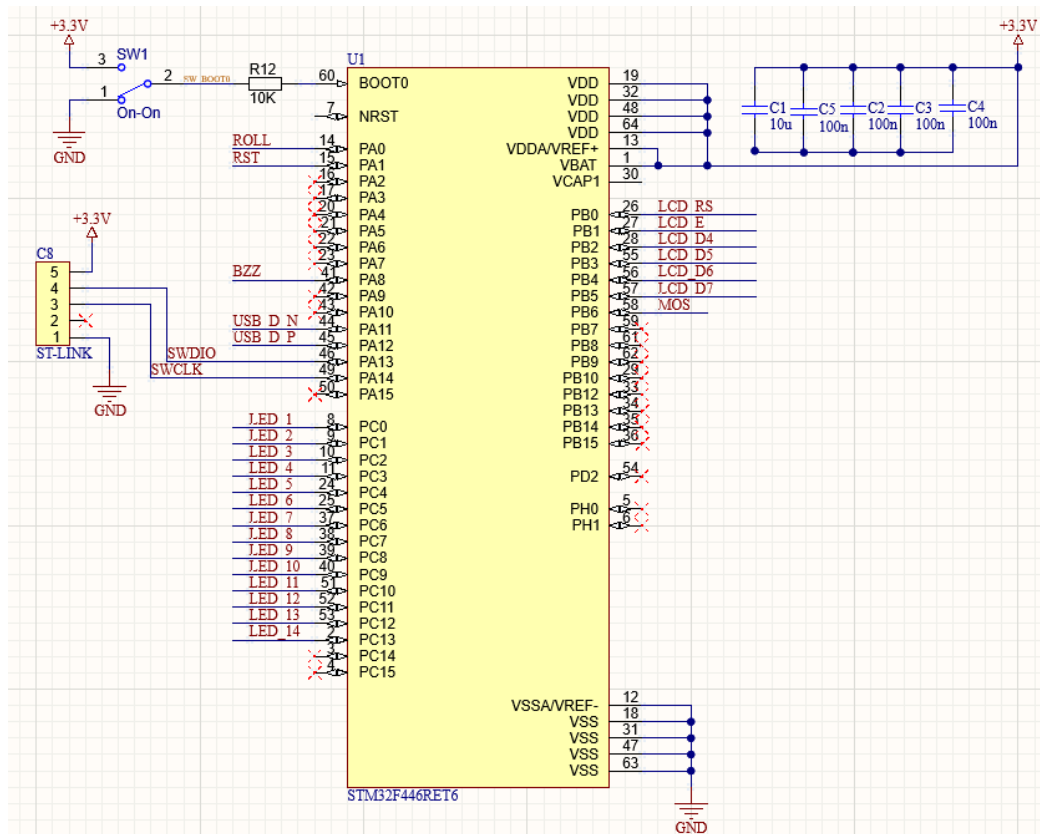


Figure 3.5: STM32 MCU Core Connections

3.4 LEDs arranged in 7-pip pattern

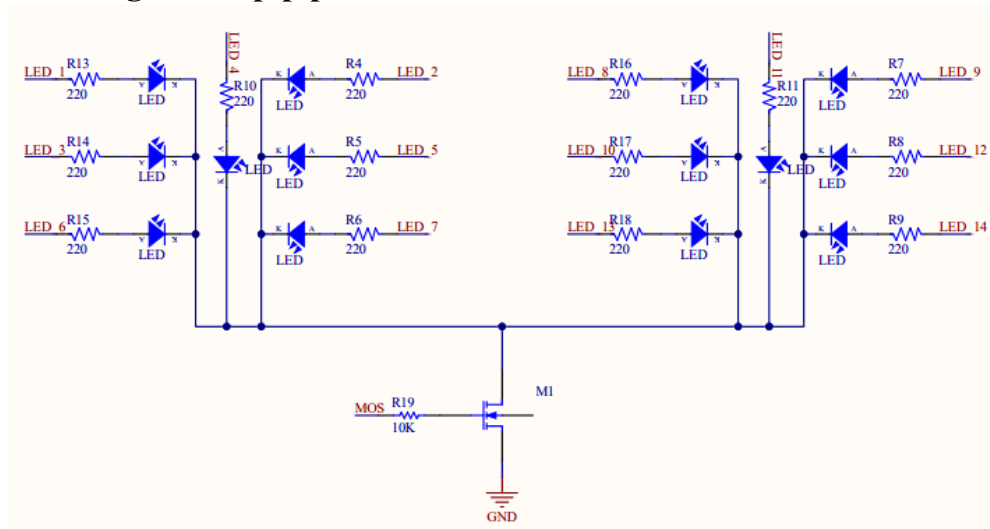


Figure 3.6: 14 LEDs arranged into pattern of two dices

All 14 LEDs (7 LEDs per dice) are arranged in the shape of 7-pip pattern that present the number and the position of all dots of a dice. Each LED is connected in series with a 220Ω current-limiting resistor (R4–R18) to protect the LED from excessive current during operation. The LEDs are powered by separate signal lines. In the center of each cluster is a central LED with resistors R10 and R11. All LED branches are connected to the Drain terminal of the MOSFET transistor M1, acting as

a low-side switching device. When the MOS control signal is applied to the MOSFET Gate via resistor R19 (10 k Ω), the MOSFET conducts and creates a connection to GND, allowing current to flow through the LEDs, causing them to light up. This structure allows the microcontroller to simultaneously turn multiple LEDs on/off with a single control signal, reducing the current load on the GPIO pins and increasing the system's power control capability.

3.5 LCD 1602 and Buzzer

The 16x2 LCD has data and control pins (RS, EN) directly connected to the microcontroller's GPIO ports. It acts as the main information interface for the game. It is responsible for displaying system messages ("SYSTEM CHECK..."), tracking turns ("PLAYER 1/2 TURN"), updating scores, and announcing the final result (Win, Loss, Draw).

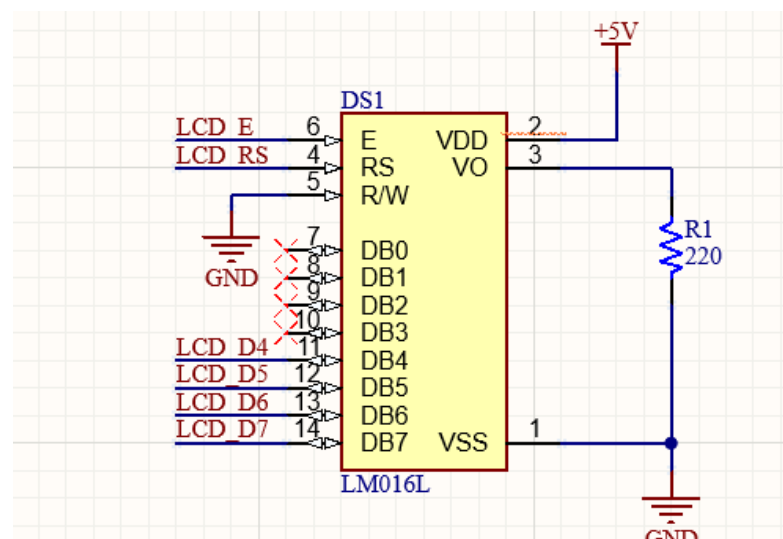


Figure 3.7: LCD 1602 for displaying game status

The buzzer is connected to a PWM-generating pin of the STM32 (e.g., pin PC12). It provides audio feedback to enhance interaction. By changing the PWM frequency, this block will produce various 8-bit sound effects such as: a "beep beep" sound when the dice are rolling, a celebratory melody for Auto-Win, or a bass sound signaling Auto-Loss.

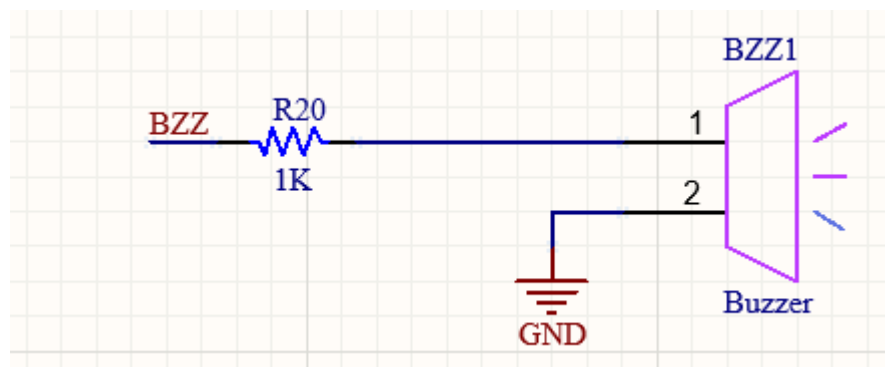


Figure 3.8: Buzzer for creating sound effect for gameplay

3.6 Hardware Implementation

The design of the circuit schematic and PCB board was done using Altium. The PCB is quite simple so that it only has two layers and the component placement is based on the classic game console, the USB port on the left side of the board is source receiving power from another USB port of Computer or Laptop.

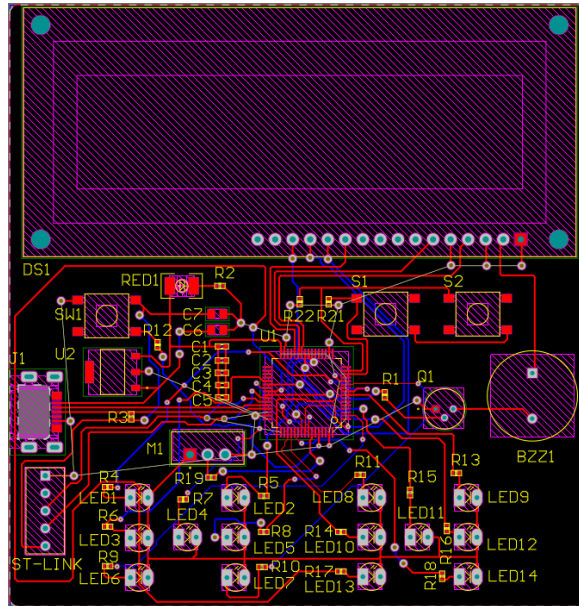


Figure 3.9: PCB layout

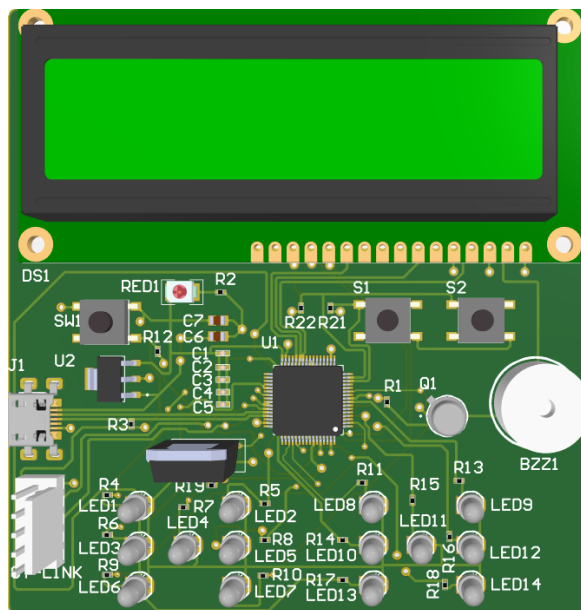


Figure 3.10: PCB layout in 3D mode

Chapter 4 Software design

4.1 Full program

The program for the system is written in C using STM32CubeIDE and import into MCU via USP port that includes ST-Link. Here is a full program that has been import into the system.

```
#include "main.h"
#include <stdlib.h>
#include <stdio.h>

uint8_t gameStep = 0;
uint8_t p1_sum = 0, p2_sum = 0;
uint8_t p1_status = 0, p2_status = 0;

const uint16_t PATTERN_CC[] = { 0x0000, 0x0008, 0x0041, 0x002A, 0x0063,
0x006B, 0x0077 };
const uint16_t PATTERN_CA[] = { 0x007F, 0x0077, 0x003E, 0x0055, 0x001C,
0x0014, 0x0008 };

#define PATTERN PATTERN_CC

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
void Custom_Delay(uint32_t ms);
void Display_All_Dice(uint8_t valA, uint8_t valB);
void Lcd_Init(void);
void Lcd_Print(char* str, uint8_t line);
void Play_End_Sound(int8_t result);

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    Lcd_Init();

    Lcd_Print("SYSTEM CHECK...", 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);

    GPIOC->ODR = 0x3FFF;
    Custom_Delay(1000);

    Display_All_Dice(0, 0);

    Lcd_Print("== DICE GAME ==", 0);
    Lcd_Print("P1 Turn: ROLL", 1);
```

```

while (1) {
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET) {
        Custom_Delay(50);
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET) {
            if (gameStep < 2) {
                srand(SysTick->VAL);
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);

                while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET) {
                    Display_All_Dice((rand() % 6) + 1, (rand() % 6) + 1);
                    Custom_Delay(60);
                }
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);

                srand(SysTick->VAL);
                uint8_t r1 = (rand() % 6) + 1;
                uint8_t r2 = (rand() % 6) + 1;
                Display_All_Dice(r1, r2);

                int8_t cur_stat = 0;
                if (r1 == r2) cur_stat = (r1==1 || r1==6) ? -1 : 1;

                if (gameStep == 0) {
                    p1_sum = r1 + r2; p1_status = cur_stat;
                    char buf[16]; sprintf(buf, "P1:%d %s", p1_sum,
(p1_status==1?"(W)":(p1_status==-1?"(L)": "")));
                    Lcd_Print(buf, 0); Lcd_Print("P2 Turn: ROLL", 1);
                    gameStep = 1;
                } else {
                    p2_sum = r1 + r2; p2_status = cur_stat;
                    char buf[16]; sprintf(buf, "P2:%d %s", p2_sum,
(p2_status==1?"(W)":(p2_status==-1?"(L)": "")));
                    Lcd_Print(buf, 1); Custom_Delay(1000);

                    int8_t res = 0;
                    if (p1_status == -1 && p2_status == -1) res = 0;
                    else if (p1_status == -1) res = 2; else if (p2_status == -1) res = 1;
                    else if (p1_status == 1 && p2_status == 1) res = 0;
                    else if (p1_status == 1) res = 1; else if (p2_status == 1) res = 2;
                    else { if (p1_sum > p2_sum) res = 1; else if (p2_sum > p1_sum) res = 2; else
res = 0; }

                    Lcd_Print(res == 0 ? "MATCH DRAW!" : (res == 1 ? "PLAYER 1 WINS!" :
"PLAYER 2 WINS!"), 0);
                    Play_End_Sound(res); Lcd_Print("RESET TO REPLAY", 1);
                    gameStep = 2;
                }
            }
        }
    }
}

```

```

    }
    }
    while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET)
Custom_Delay(10);
    }
    }

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET) {
        Custom_Delay(50);
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET) {
            gameStep = 0;
            Display_All_Dice(0, 0);
            Lcd_Print("== NEW MATCH ==", 0); Lcd_Print("P1 Turn: ROLL", 1);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
Custom_Delay(100);
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
            while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET)
Custom_Delay(10);
        }
    }
}

void Display_All_Dice(uint8_t valA, uint8_t valB) {
    uint32_t out = (PATTERN[valA] & 0x7F) | ((uint32_t)(PATTERN[valB] & 0x7F)
<< 7);
    GPIOC->ODR = out;

    if (valA > 0 || valB > 0) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
    } else {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
    }
}

static void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStructure.Pin = 0x3FFF;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStructure);
}

```

```

    GPIO_InitStruct.Pin =
GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5|G
PIO_PIN_6;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = GPIO_PIN_0 | GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = GPIO_PIN_8;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    __HAL_RCC_PWR_CLK_ENABLE();

    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_S
CALE3);
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);
    RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK|RCC_CLOCKTYPE_PC
LK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYCLKSource = RCC_SYCLKSOURCE_HSI;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0);
}

void Lcd_Write(uint8_t data, uint8_t rs) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, rs);
    GPIOB->ODR = (GPIOB->ODR & ~0x3C) | ((data & 0x0F) << 2);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, 1); Custom_Delay(1);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, 0); Custom_Delay(1);
}

void Lcd_Init(void) {

```

```

Custom_Delay(50); Lcd_Write(0x03, 0); Custom_Delay(5);
Lcd_Write(0x03, 0); Custom_Delay(1); Lcd_Write(0x03, 0); Lcd_Write(0x02, 0);
Lcd_Write(0x2, 0); Lcd_Write(0x8, 0); Lcd_Write(0x0, 0); Lcd_Write(0xC, 0);
Lcd_Write(0x0, 0); Lcd_Write(0x1, 0); Custom_Delay(2);
}
void Lcd_Print(char* str, uint8_t line) {
    Lcd_Write((line == 0 ? 0x80 : 0xC0) >> 4, 0); Lcd_Write(line == 0 ? 0x0 : 0x0, 0);
    Lcd_Write((line == 0 ? 0x80 : 0xC0) >> 4, 0); Lcd_Write((line == 0 ? 0x80 :
0xC0), 0);
    uint8_t count = 0;
    while(*str && count < 16) {
        Lcd_Write((*str) >> 4, 1);
        Lcd_Write(*str, 1);
        str++;
        count++;
    }
    while(count < 16) {
        Lcd_Write(0x20 >> 4, 1);
        Lcd_Write(0x20, 1);
        count++;
    }
}
void Play_End_Sound(int8_t result) {
    if (result == 0) {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, 1); Custom_Delay(500);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, 0);
    } else if (result == 1) {
        for(int i=0; i<2; i++) { HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, 1);
Custom_Delay(200); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, 0);
Custom_Delay(200); }
    } else if (result == 2) {
        for(int i=0; i<4; i++) { HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, 1);
Custom_Delay(50); HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, 0);
Custom_Delay(50); }
    }
}
void Custom_Delay(uint32_t ms) { for(volatile uint32_t i=0; i < ms * 1600; i++)
    NOP(); }

```

Chapter 5 Result

Please check out the video of implementation via the link below:

https://drive.google.com/file/d/1LmK-j6QYU7Z-uB2F_15ujnoRMx5scoLS/view?usp=sharing

Or via QR link below:

